# 키 관리 및 보안

## 개요

CometBFT의 키 관리 시스템은 밸리데이터의 프라이빗 키를 안전하게 보관하고 관리하는 핵심 보안 구성 요소입니다. 이 시스템은 이중 서명 방지, 키 보안, 원격 서명 등의 기능을 통해 블록체인 네트워크의 보안을 보장합니다.

CometBFT는 다양한 키 관리 방식을 지원합니다:

- **파일 기반 키 관리**: 로컬 파일 시스템에 키 저장
- **원격 서명자**: 별도의 보안 환경에서 키 관리
- **하드웨어 보안 모듈**: HSM을 통한 키 보호
- **이중 서명 방지**: 악의적인 행동 탐지 및 방지

## 핵심 개념

### 키 관리 구성 요소

1. **PrivValidator 인터페이스**

- 밸리데이터의 서명 기능을 추상화
- 다양한 구현체 지원 (파일, 원격, HSM)

2. **키 저장소 (Key Store)**

- 프라이빗 키의 안전한 저장
- 접근 권한 제어

3. **상태 저장소 (State Store)**

- 마지막 서명 정보 추적
- 이중 서명 방지

4. **원격 서명자 (Remote Signer)**

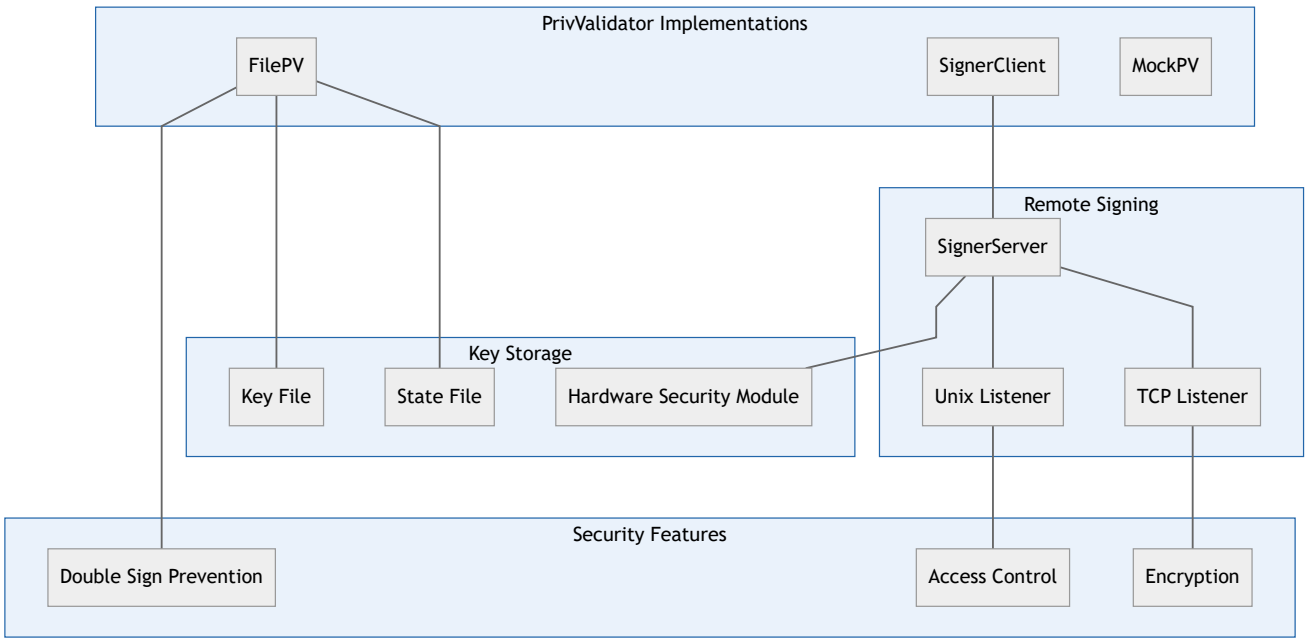
- 네트워크를 통한 안전한 서명
- 키 격리 및 보안 강화

### 보안 원칙

- **최소 권한 원칙**: 필요한 최소한의 권한만 부여
- **키 격리**: 프라이빗 키를 별도 환경에서 관리
- **이중 서명 방지**: 동일한 높이/라운드에서 중복 서명 차단
- **감사 추적**: 모든 서명 활동 기록

## 아키텍처

### 키 관리 시스템 구조




## PrivValidator 인터페이스

```go
type PrivValidator interface {

// 공개키 반환

GetPubKey() (crypto.PubKey, error)

// 투표 서명

SignVote(chainID  string, vote *cmtproto.Vote, signExtension  bool) error

// 블록 제안 서명

SignProposal(chainID  string, proposal *cmtproto.Proposal) error

// 임의 바이트 서명

SignBytes(bytes []byte) ([]byte, error)
```

```
}
```

## 파일 기반 키 관리

### FilePV 구조

FilePV는 로컬 파일 시스템을 사용하는 기본 키 관리 구현체입니다:

```
// 불변 키 정보
type FilePVKey struct {
Address types.Address `json:"address"`
PubKey crypto.PubKey `json:"pub_key"`
PrivKey crypto.PrivKey `json:"priv_key"`
filePath  string
}

// 가변 상태 정보
type FilePVLastSignState struct {
Height  int64  `json:"height"`
Round  int32  `json:"round"`
Step  int8  `json:"step"`
Signature []byte  `json:"signature,omitempty"`
SignBytes cmtbytes.HexBytes `json:"signbytes,omitempty"`
filePath  string
```

```
}

// FilePV 구조체
type FilePV struct {
Key FilePVKey
LastSignState FilePVLastSignState
}
```

## 키 생성 및 로딩

```
// 새로운 키 생성
func GenFilePV(keyFilePath, stateFilePath string, keyGen func() (crypto.PrivK
if keyGen == nil {
keyGen = func() (crypto.PrivKey, error) {
return ed25519.GenPrivKey(), nil
}
}
key, err := keyGen()
if err != nil {
return nil, err
}
return NewFilePV(key, keyFilePath, stateFilePath), nil
}
```

```go
// 기존 키 로딩
func LoadFilePV(keyFilePath, stateFilePath string) *FilePV {
// 키 파일 읽기
keyJSONBytes, err := os.ReadFile(keyFilePath)
if err != nil {
cmtos.Exit(err.Error())
}
pvKey := FilePVKey{}
err = cmtjson.Unmarshal(keyJSONBytes, &pvKey)
if err != nil {
cmtos.Exit(fmt.Sprintf("Error reading PrivValidator key from %v: %v\n", keyFileP
}
// 공개키와 주소 재계산
pvKey.PubKey = pvKey.PrivKey.PubKey()
pvKey.Address = pvKey.PubKey.Address()
pvKey.filePath = keyFilePath
// 상태 파일 읽기
pvState := FilePVLastSignState{}
stateJSONBytes, err := os.ReadFile(stateFilePath)
if err != nil {
cmtos.Exit(err.Error())
}
```

```
err = cmtjson.Unmarshal(stateJSONBytes, &pvState)
if err != nil {
cmtos.Exit(fmt.Sprintf("Error reading PrivValidator state from %v: %v\n", stateF
}
pvState.filePath = stateFilePath
return &FilePV{
Key: pvKey,
LastSignState: pvState,
}
}


// 키 존재 여부에 따른 로딩/생성
func LoadOrGenFilePV(keyFilePath, stateFilePath string, keyGenF func() (crypt
var pv *FilePV
if cmtos.FileExists(keyFilePath) {
pv = LoadFilePV(keyFilePath, stateFilePath)
} else {
var err error
pv, err = GenFilePV(keyFilePath, stateFilePath, keyGenF)
if err != nil {
return nil, err
}
pv.Save()
```

```
}
return pv, nil
}
```

## 안전한 파일 저장

```
// 키 파일 저장
func (pvKey FilePVKey) Save() {
outFile := pvKey.filePath
if outFile == "" {
panic("cannot save PrivValidator key: filePath not set")
}
jsonBytes, err := cmtjson.MarshalIndent(pvKey, "", " ")
if err != nil {
panic(err)
}
// 원자적 파일 쓰기 (0600 권한)
if err := tempfile.WriteFileAtomic(outFile, jsonBytes, 0o600); err != nil {
panic(err)
}
}
```

```go
// 상태 파일 저장
func (lss *FilePVLastSignState) Save() {
outFile := lss.filePath
if  outFile == "" {
panic("cannot save FilePVLastSignState: filePath not set")
}
jsonBytes, err := cmtjson.MarshalIndent(lss, "", " ")
if  err != nil {
panic(err)
}
// 원자적 파일 쓰기
err = tempfile.WriteFileAtomic(outFile, jsonBytes, 0o600)
if  err != nil {
panic(err)
}
}
```

# 이중 서명 방지

## HRS (Height-Round-Step) 검증

이중 서명 방지의 핵심은 HRS 검증입니다:

```go
// HRS 검증 함수
```

```go
func (lss *FilePVLastSignState) CheckHRS(height int64, round int32, step int8
// 높이 회귀 검사
if lss.Height > height {
return false, fmt.Errorf("height regression. Got %v, last height %v", height, ls
}
if lss.Height != height {
return false, nil // 새로운 높이
}
// 라운드 회귀 검사
if lss.Round > round {
return false, fmt.Errorf("round regression at height %v. Got %v, last round %v",
height, round, lss.Round)
}
if lss.Round != round {
return false, nil // 새로운 라운드
}
// 단계 회귀 검사
if lss.Step > step {
return false, fmt.Errorf("step regression at height %v round %v. Got %v, last st
height, round, step, lss.Step)
}
if lss.Step < step {
return false, nil // 새로운 단계
```

```go
}
// 동일한 HRS - 서명 바이트 확인
if lss.SignBytes == nil {
return false, errors.New("no SignBytes found")
}
if lss.Signature == nil {
panic("pv: Signature is nil but SignBytes is not!")
}
return true, nil // 재사용 가능
}
```

## 안전한 투표 서명

```go
func (pv *FilePV) signVote(chainID string, vote *cmtproto.Vote, signExtension
height, round, step := vote.Height, vote.Round, voteToStep(vote)
lss := pv.LastSignState
// HRS 검증
sameHRS, err := lss.CheckHRS(height, round, step)
if err != nil {
return err
}
signBytes := types.VoteSignBytes(chainID, vote)
// 확장 서명 처리
```

```
if signExtension {
var extSig, nonRpExtSig []byte
if vote.Type == types.PrecommitType && !types.ProtoBlockIDIsNil(&vote.BlockID)
extSignBytes, nonRpExtSignBytes := types.VoteExtensionSignBytes(chainID, vote)
extSig, err = pv.Key.PrivKey.Sign(extSignBytes)
if err != nil {
return err
}
nonRpExtSig, err = pv.Key.PrivKey.Sign(nonRpExtSignBytes)
if err != nil {
return err
}
} else if len(vote.Extension) > 0 || len(vote.NonRpExtension) > 0 {
return errors.New("unexpected vote extension - extensions are only allowed in n
}
vote.ExtensionSignature = extSig
vote.NonRpExtensionSignature = nonRpExtSig
}
// 동일한 HRS인 경우 기존 서명 재사용 검토
if sameHRS {
if bytes.Equal(signBytes, lss.SignBytes) {
// 완전히 동일한 서명 바이트 - 재사용
vote.Signature = lss.Signature
```

```
} else  if  timestamp, ok := checkVotesOnlyDifferByTimestamp(lss.SignBytes, sign

// 타임스탬프만 다른 경우 - 기존 타임스탬프와 서명 사용

vote.Timestamp = timestamp

vote.Signature = lss.Signature

} else {

// 충돌하는 데이터 - 이중 서명 시도

err = errors.New("conflicting data")

}

return  err

}

// 새로운 서명 생성

sig, err := pv.Key.PrivKey.Sign(signBytes)

if  err != nil {

return  err

}

// 서명 정보 저장

pv.saveSigned(height, round, step, signBytes, sig)

vote.Signature = sig

return  nil

}
```

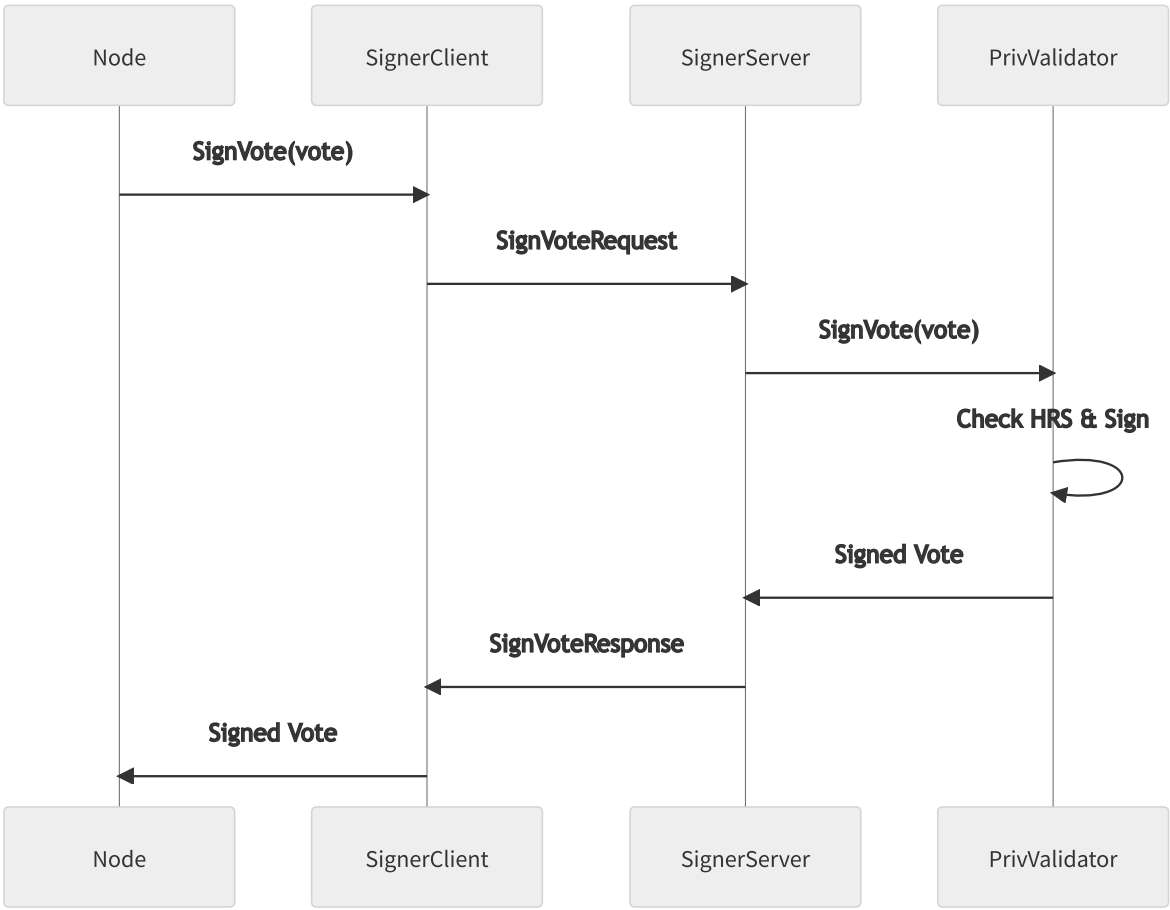
## 타임스탬프 차이 검증

```go
// 투표가 타임스탬프만 다른지 확인
func checkVotesOnlyDifferByTimestamp(lastSignBytes, newSignBytes []byte) (time.
var lastVote, newVote cmtproto.CanonicalVote
if err := protoio.UnmarshalDelimited(lastSignBytes, &lastVote); err != nil {
panic(fmt.Sprintf("LastSignBytes cannot be unmarshalled into vote: %v", err))
}
if err := protoio.UnmarshalDelimited(newSignBytes, &newVote); err != nil {
panic(fmt.Sprintf("signBytes cannot be unmarshalled into vote: %v", err))
}
lastTime := lastVote.Timestamp
// 타임스탬프를 동일하게 설정하고 비교
now := cmttime.Now()
lastVote.Timestamp = now
newVote.Timestamp = now
return lastTime, proto.Equal(&newVote, &lastVote)
}
```

# 원격 서명자

## 원격 서명 아키텍처

원격 서명자는 키를 별도의 보안 환경에서 관리합니다:




## SignerClient 구현

```
type SignerClient struct {
endpoint *SignerListenerEndpoint
chainID  string
}

// 원격 서명자에서 공개키 가져오기
func (sc *SignerClient) GetPubKey() (crypto.PubKey, error) {
```

```go
response, err := sc.endpoint.SendRequest(mustWrapMsg(&pvproto.PubKeyRequest{Chai
if err != nil {
return nil, fmt.Errorf("send: %w", err)
}
resp := response.GetPubKeyResponse()
if resp == nil {
return nil, cmterrors.ErrRequiredField{Field: "response"}
}
if resp.Error != nil {
return nil, &RemoteSignerError{Code: int(resp.Error.Code), Description: resp.Er
}
pk, err := cryptoenc.PubKeyFromTypeAndBytes(resp.PubKeyType, resp.PubKeyBytes)
if err != nil {
return nil, err
}
return pk, nil
}

// 원격 투표 서명
func (sc *SignerClient) SignVote(chainID string, vote *cmtproto.Vote, signExten
response, err := sc.endpoint.SendRequest(mustWrapMsg(&pvproto.SignVoteRequest{
Vote: vote,
ChainId: chainID,
```

```
SkipExtensionSigning: !signExtension,
}))
if err != nil {
return err
}
resp := response.GetSignedVoteResponse()
if resp == nil {
return cmterrors.ErrRequiredField{Field: "response"}
}
if resp.Error != nil {
return &RemoteSignerError{Code: int(resp.Error.Code), Description: resp.Error.De
}
*vote = resp.Vote
return nil
}
```

## SignerServer 구현

```
type SignerServer struct {
service.BaseService
endpoint *SignerDialerEndpoint
chainID string
privVal types.PrivValidator
```

```
handlerMtx cmtsync.Mutex

validationRequestHandler ValidationRequestHandlerFunc

}



// 서명 요청 처리

func (ss *SignerServer) servicePendingRequest() {

if !ss.IsRunning() {

return

}

req, err := ss.endpoint.ReadMessage()

if  err != nil {

if  err != io.EOF {

ss.Logger.Error("SignerServer: HandleMessage", "err", err)

}

return

}

var  res privvalproto.Message

func() {

ss.handlerMtx.Lock()

defer  ss.handlerMtx.Unlock()

res, err = ss.validationRequestHandler(ss.privVal, req, ss.chainID)

if  err != nil {

ss.Logger.Error("SignerServer: handleMessage", "err", err)
```

```
}
}()
err = ss.endpoint.WriteMessage(res)
if err != nil {
ss.Logger.Error("SignerServer: writeMessage", "err", err)
}
}
```

## 네트워크 보안

### TCP 연결 보안

TCP 기반 원격 서명에서는 암호화된 연결을 사용합니다:

```
// TCP 다이얼러
func DialTCPFn(addr string, timeoutReadWrite time.Duration, privKey crypto.Pri
return func() (net.Conn, error) {
conn, err := cmtnet.Connect(addr)
if err == nil {
deadline := time.Now().Add(timeoutReadWrite)
err = conn.SetDeadline(deadline)
}
if err == nil {
// 암호화된 연결 생성
```

```go
conn, err = p2pconn.MakeSecretConnection(conn, privKey)
}
return  conn, err
}
}

// TCP 리스너
type TCPListener struct {
*net.TCPListener
secretConnKey ed25519.PrivKey
timeoutAccept time.Duration
timeoutReadWrite time.Duration
}

func (ln *TCPListener) Accept() (net.Conn, error) {
deadline := time.Now().Add(ln.timeoutAccept)
err := ln.SetDeadline(deadline)
if  err != nil {
return  nil, err
}
tc, err := ln.AcceptTCP()
if  err != nil {
return  nil, err
```

```
}

// 타임아웃 래퍼

timeoutConn := newTimeoutConn(tc, ln.timeoutReadWrite)

// 암호화된 연결 생성

secretConn, err := p2pconn.MakeSecretConnection(timeoutConn, ln.secretConnKey)

if  err != nil {

return  nil, err

}

return  secretConn, nil

}
```

### Unix 소켓 보안

Unix 소켓은 파일 시스템 권한을 통해 보안을 제공합니다:

```
type UnixListener struct {

*net.UnixListener

timeoutAccept time.Duration

timeoutReadWrite time.Duration

}


func (ln *UnixListener) Accept() (net.Conn, error) {

deadline := time.Now().Add(ln.timeoutAccept)
```

```
err := ln.SetDeadline(deadline)
if err != nil {
return nil, err
}
tc, err := ln.AcceptUnix()
if err != nil {
return nil, err
}
// 타임아웃 래퍼만 적용 (암호화 없음)
conn := newTimeoutConn(tc, ln.timeoutReadWrite)
return conn, nil
}
```

## 키 백업 및 복구

### 키 백업 전략

```
// 키 백업 생성
func BackupPrivValidator(pv *FilePV, backupPath string) error {
// 키 정보 백업
keyBackup := struct {
Address string `json:"address"`
PubKey string `json:"pub_key"`
```

```
PrivKey  string  `json:"priv_key"`
}{
Address: pv.Key.Address.String(),
PubKey: fmt.Sprintf("%X", pv.Key.PubKey.Bytes()),
PrivKey: fmt.Sprintf("%X", pv.Key.PrivKey.Bytes()),
}
jsonBytes, err := cmtjson.MarshalIndent(keyBackup, "", " ")
if  err != nil {
return  err
}
// 백업 파일 생성 (읽기 전용)
return  tempfile.WriteFileAtomic(backupPath, jsonBytes, 0o400)
}


// 키 복구
func  RestorePrivValidator(backupPath, keyPath, statePath  string) (*FilePV, err
backupBytes, err := os.ReadFile(backupPath)
if  err != nil {
return  nil, err
}
var  keyBackup  struct {
Address  string  `json:"address"`
PubKey  string  `json:"pub_key"`
```

```
PrivKey  string  `json:"priv_key"`
}
err = cmtjson.Unmarshal(backupBytes, &keyBackup)
if  err != nil {
return  nil, err
}
// 프라이빗 키 복원
privKeyBytes, err := hex.DecodeString(keyBackup.PrivKey)
if  err != nil {
return  nil, err
}
privKey := ed25519.PrivKey(privKeyBytes)
// 새로운 FilePV 생성
pv := NewFilePV(privKey, keyPath, statePath)
pv.Save()
return  pv, nil
}
```

## 키 순환 (Key Rotation)

```
// 키 순환 프로세스
func  RotateValidatorKey(oldPV *FilePV, newKeyPath, newStatePath  string) (*File
// 새로운 키 생성
```

```
newPV, err := GenFilePV(newKeyPath, newStatePath, nil)
if err != nil {
return nil, err
}
// 이전 상태 정보 복사 (선택적)
newPV.LastSignState.Height = oldPV.LastSignState.Height
newPV.LastSignState.Round = oldPV.LastSignState.Round
newPV.LastSignState.Step = oldPV.LastSignState.Step
// 새로운 키 저장
newPV.Save()
return newPV, nil
}
```

## 보안 모범 사례

### 파일 권한 설정

```
# 키 파일 권한 (소유자만 읽기)
chmod 600 priv_validator_key.json

# 상태 파일 권한 (소유자만 읽기/쓰기)
chmod 600 priv_validator_state.json
```

```
# 디렉토리 권한 (소유자만 접근)
chmod 700 ~/.cometbft/config/
```

## 환경 분리

```yaml
# Docker Compose 예제
version: '3.8'
services:
cometbft-node:
image: cometbft/cometbft
volumes:
- ./config:/cometbft/config
environment:
- CMTPRIV_VALIDATOR_LADDR=tcp://0.0.0.0:26659
remote-signer:
image: cometbft/remote-signer
volumes:
- ./signer-config:/signer/config
environment:
- SIGNER_CHAIN_ID=test-chain
- SIGNER_LISTEN_ADDR=tcp://0.0.0.0:26659
networks:
- signer-network
```

```yaml
networks:
signer-network:
driver: bridge
internal: true
```

## 모니터링 및 알림

```go
// 서명 활동 모니터링
type SigningMonitor struct {
logger log.Logger
metrics *Metrics
}

func (sm *SigningMonitor) LogSigningActivity(height  int64, round  int32, step
sm.logger.Info("Signing activity",
"height", height,
"round", round,
"step", step,
"success", success)
if  success {
sm.metrics.SigningSuccessCount.Inc()
} else {
```

```
sm.metrics.SigningFailureCount.Inc()

// 실패 시 알림 발송

sm.sendAlert("Signing failed", height, round, step)

}

}


func (sm *SigningMonitor) sendAlert(message  string, height  int64, round  int32

// 알림 시스템 연동 (Slack, PagerDuty 등)

alert := Alert{

Message: message,

Height: height,

Round: round,

Step: step,

Time: time.Now(),

}

// 알림 발송 로직

sm.sendToAlertManager(alert)

}
```

## 정기 보안 점검

```
// 보안 점검 체크리스트

type SecurityAudit struct {
```

```
KeyFilePermissions  bool

StateFilePermissions  bool

BackupExists  bool

LastSigningActivity time.Time

DoubleSignAttempts  int

}



func (sa *SecurityAudit) PerformAudit(pv *FilePV) error {

// 파일 권한 확인

sa.KeyFilePermissions = sa.checkFilePermissions(pv.Key.filePath, 0o600)

sa.StateFilePermissions = sa.checkFilePermissions(pv.LastSignState.filePath, 0o6

// 백업 존재 확인

sa.BackupExists = sa.checkBackupExists(pv.Key.filePath + ".backup")

// 마지막 서명 활동 확인

sa.LastSigningActivity = sa.getLastSigningTime(pv)

// 이중 서명 시도 확인

sa.DoubleSignAttempts = sa.countDoubleSignAttempts(pv)

return  sa.generateReport()

}
```

CometBFT의 키 관리 시스템은 이러한 다층적인 보안 메커니즘을 통해 밸리데이터의 프라이빗 키를 안전하게 보호하고, 블록체인 네트워크의 보안과 신뢰성을 보장합니다.